

Week 3: Seller and Inventory Management System

Scenario:

An online shopping platform deals with products received from different sellers. To manage this information effectively, a database is required to store seller details and maintain information about the products they provide. The system also needs to keep track of the quantity of products currently available in stock. This makes it easier to identify available products and monitor inventory levels.

Database Tables:

The system is implemented using the following tables:

Seller Table:

This table contains the basic information of sellers registered with the e-commerce platform.

```
SQL> CREATE TABLE SELLER(SELLER_ID VARCHAR2(7),SELLER_NAME VARCHAR2(25),EMAIL VARCHAR2(30),PHONE_NUMBER VARCHAR2(10),ADDRESS VARCHAR2(45));  
Table created.
```

Inventory Table:

This table records the stock information related to the products available in the system.

```
SQL> CREATE TABLE INVENTORY(INVENTORY_ID VARCHAR2(7),PRODUCT_ID VARCHAR2(7),SELLER_ID VARCHAR2(7),STOCK_QUANTITY NUMBER(11),STOCK_STATUS VARCHAR2(20),LAST_UP  
DATED DATE);  
Table created.
```

Relationship Between Entities

Seller – Product:

A single seller can provide several products to the online store. Similarly, a product can be associated with different sellers.

Cardinality: One-to-Many

Seller → Product

Product – Inventory

Inventory details are maintained for each product. The inventory record is used to store the quantity currently available.

Cardinality: One-to-One

Product → Inventory

Keys Used in the Database

To identify records and connect related information, keys are defined in the tables.

- A **Primary Key** uniquely identifies each record.

```
SQL> ALTER TABLE Seller
  2  ADD CONSTRAINT PK_Seller
  3  PRIMARY KEY (Seller_ID);
```

Table altered.

Primary Key for Inventory Table

```
SQL> ALTER TABLE Inventory
  2  ADD CONSTRAINT PK_Inventory
  3  PRIMARY KEY (Inventory_ID);
```

Table altered.

Foreign Keys

A **Foreign Key** connects a record in the Inventory table with its corresponding product.

These keys help maintain proper relationships between the tables.

Product Foreign Key

```
SQL> ALTER TABLE Inventory
  2  ADD CONSTRAINT FK_Inventory_Product
  3  FOREIGN KEY (Product_ID)
  4  REFERENCES Product(Product_ID);
```

Table altered.

Seller Foreign Key

```
SQL> ALTER TABLE Inventory
  2  ADD CONSTRAINT FK_Inventory_Seller
  3  FOREIGN KEY (Seller_ID)
  4  REFERENCES Seller(Seller_ID);

Table altered.
```

This establishes the connection between **Product–Inventory** and **Seller–Inventory**.

Constraints Applied

Constraints are added to prevent invalid or duplicate data from being entered into the database.

Make Email UNIQUE

```
SQL> ALTER TABLE Seller
  2  ADD CONSTRAINT UQ_Seller_Email
  3  UNIQUE (Email);

Table altered.
```

This ensures that two sellers cannot have the same email ID.

```
SQL> ALTER TABLE Seller
  2  MODIFY Email NOT NULL;

Table altered.
```

This ensures that every seller must have an email ID.

Check Stock Quantity is Positive

```
SQL> ALTER TABLE Inventory
  2  ADD CONSTRAINT CK_Stock_Quantity
  3  CHECK (Stock_Quantity >= 0);

Table altered.
```

This prevents negative stock quantities.

Adding Data to the Tables

After creating the tables and applying the required constraints, sample records are entered into the database.

Seller details are inserted into the Seller table, while the corresponding product stock information is entered into the Inventory table.

This data is later used to test different SQL operations and generate reports.

```
SQL> INSERT INTO Seller
  2 VALUES (
  3     'S00001',
  4     'Vani Enterprises',
  5     'mdvanienter@gmail.com',
  6     '8765432109',
  7     '1, Anna Nagar, Chennai'
  8 );
```

1 row created.

```
SQL>
SQL> INSERT INTO Seller
  2 VALUES (
  3     'S00002',
  4     'SD & CO',
  5     'seller0827@gmail.com',
  6     '9876543012',
  7     '62, Thirumal Nagar, Thanjavur'
  8 );
```

Enter value for co:

```
old  4:      'SD & CO',
new  4:      'SD ',
```

1 row created.

```
SQL> INSERT INTO Seller
  2 VALUES (
  3     'S00003',
  4     'AtoZ',
  5     'atozmarket@gmail.com',
  6     '7890654312',
  7     '90, George Town, Chennai'
  8 );
```

1 row created.

```
SQL>
SQL> INSERT INTO Seller
  2 VALUES (
  3     'S00004',
  4     'DSW Manufacturers',
  5     'dsvmanuf142@gmail.com',
  6     '7824003009',
  7     '23, Thillai Nagar, Trichy'
  8 );
```

1 row created.

```
SQL>
SQL> COMMIT;
```

Commit complete.

```
SQL> SELECT * FROM Seller;
```

SELLER_	SELLER_NAME	EMAIL	PHONE_NUMB
---------	-------------	-------	------------

ADDRESS

S00001	Vani Enterprises 1, Anna Nagar, Chennai	mdvanienter@gmail.com	8765432109
--------	--	-----------------------	------------

S00002	SD 62, Thirumal Nagar, Thanjavur	seller0827@gmail.com	9876543012
--------	-------------------------------------	----------------------	------------

S00003	AtoZ 90, George Town, Chennai	atozmarket@gmail.com	7890654312
--------	----------------------------------	----------------------	------------

SELLER_	SELLER_NAME	EMAIL	PHONE_NUMB
---------	-------------	-------	------------

ADDRESS

S00004	DSW Manufacturers 23, Thillai Nagar, Trichy	dsvmanuf142@gmail.com	7824003009
--------	--	-----------------------	------------

Insert Values into Inventory Table

```
SQL> INSERT INTO Inventory
  2  VALUES (
  3      'INV0001',
  4      'P100001',
  5      'S00001',
  6      4,
  7      'Available',
  8      TO_DATE('2026-09-12', 'YYYY-MM-DD')
  9  );
```

1 row created.

```
SQL>
SQL> INSERT INTO Inventory
  2  VALUES (
  3      'INV0002',
  4      'P100001',
  5      'S00002',
  6      3,
  7      'Available',
  8      TO_DATE('2026-08-13', 'YYYY-MM-DD')
  9  );
```

1 row created.

```
SQL>
SQL> INSERT INTO Inventory
  2  VALUES (
  3      'INV0003',
  4      'P100001',
  5      'S00003',
  6      4,
  7      'Unavailable',
  8      TO_DATE('2026-09-15', 'YYYY-MM-DD')
  9  );
```

1 row created.

```
SQL>
SQL> INSERT INTO Inventory
  2  VALUES (
  3      'INV0004',
  4      'P200002',
  5      'S00004',
  6      3,
  7      'Available',
  8      TO_DATE('2026-09-14', 'YYYY-MM-DD')
  9  );
```

1 row created.

```
SQL>
SQL> COMMIT;
```

Commit complete.

```
SQL> SELECT * FROM Inventory;
```

INVENTO	PRODUCT	SELLER_	STOCK_QUANTITY	STOCK_STATUS	LAST_UPDA
INV0001	P100001	S00001	4	Available	12-SEP-26
INV0002	P100001	S00002	3	Available	13-AUG-26
INV0003	P100001	S00003	4	Unavailable	15-SEP-26
INV0004	P200002	S00004	3	Available	14-SEP-26

Seller Performance and Product Supply Report

This report is prepared to examine the products supplied by individual sellers.

The report contains:

- Seller name
- Number of products provided
- Quantity of stock available

The required information is obtained by combining data from the related tables using **JOIN operations**.

```
SQL> SELECT s.Seller_Name,  
2          COUNT(i.Product_ID) AS Number_of_Products,  
3          SUM(i.Stock_Quantity) AS Available_Stock  
4 FROM Seller s  
5 JOIN Inventory i  
6 ON s.Seller_ID = i.Seller_ID  
7 WHERE i.Stock_Status = 'Available'  
8 GROUP BY s.Seller_Name;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
SD	1	3
DSW Manufacturers	1	3
Vani Enterprises	1	4

Product Stock Monitoring Report

This report focuses on the current stock position of each product.

The following details are displayed:

- Product name
- Quantity available
- Current stock condition

The stock condition can be identified based on whether the product has sufficient stock or has reached zero stock.

```
SQL> SELECT p.Product_Name,  
2         i.Stock_Quantity,  
3         i.Stock_Status  
4 FROM Product p  
5 JOIN Inventory i  
6 ON p.Product_ID = i.Product_ID;
```

PRODUCT_NAME	STOCK_QUANTITY	STOCK_STATUS
Laptop	4	Available
Laptop	3	Available
Laptop	4	Unavailable
Kids Shoes	3	Available

Inventory Statistics Report

This section provides a numerical summary of the inventory maintained by the company.

The following values are calculated:

- Overall number of products
- Number of products with no available stock
- Product with the greatest quantity in stock
- Average quantity of products available

SQL functions such as **COUNT()**, **MAX()**, and **AVG()** can be used to obtain these results.

a) Total Products Available

```
SQL> SELECT COUNT(*) AS Total_Available_Products
  2  FROM Inventory
  3  WHERE Stock_Status = 'Available';
```

```
TOTAL_AVAILABLE_PRODUCTS
-----
                        3
```

b) Products Out of Stock

```
SQL> SELECT COUNT(*) AS OutOfStock_Products
  2  FROM Inventory
  3  WHERE Stock_Status = 'Out of Stock';
```

```
OUTOFSTOCK_PRODUCTS
-----
                        0
```

c) Highest Stocked Products

```

SQL> SELECT p.Product_Name,
2       i.Stock_Quantity
3 FROM Product p
4 JOIN Inventory i
5 ON p.Product_ID = i.Product_ID
6 WHERE i.Stock_Quantity = (
7       SELECT MAX(Stock_Quantity)
8       FROM Inventory
9 );

```

PRODUCT_NAME	STOCK_QUANTITY
Laptop	4
Laptop	4

d) Average Inventory Quantity

```

SQL> SELECT AVG(Stock_Quantity) AS Average_Inventory
2 FROM Inventory;

```

AVERAGE_INVENTORY
3.5

Conclusion:

The Seller and Inventory Management System demonstrates how a relational database can be used to organize seller and stock-related information. Proper keys and constraints help maintain the correctness of the stored data, while relationships allow information from different tables to be connected. The generated reports provide a clear view of seller contributions and current inventory conditions. Thus, the database provides a structured approach for maintaining product and stock information in an e-commerce environment.